

Lecture:4

Pointer and structure

* Revision

Que:1 Why does scanf require &, to scan primitive data types?

Ans:

```
void make_it_5(int *ptr) {  
    *ptr = 5;  
}
```

```
int main() {  
    int x;  
    make_it_5(&x);  
    printf("%d", x);  
}
```

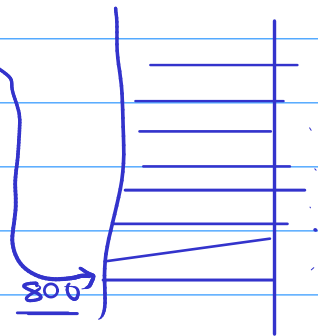
```
scanf(↓) {  
    _____  
}
```

```
int main() {  
    int x;  
    scanf("%d", &x);  
}
```

Que:2 Why & is not required to scan string?

Ans:2 char name[10];

scanf("%s", name);



Que:3: find occurrences of a string in other string using strstr.

```
char input[] = "DDDUDDDU", to_search[] = "DDU";  
// strstr(input, to_search);
```

```
char *ptr = input;
```

```
int cnt = 0;
```

```
while(1) {
```

```
    ptr = strstr(ptr, to_search);
```

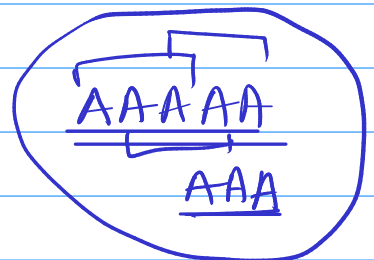
```
    if( ptr != NULL) {
```

```
        cnt++;  
        ptr++;
```

```
    }  
    else
```

```
        break;
```

```
    printf("%d", cnt);
```



Que:4 What is the difference between

<pre> int sum(int <u>arr</u>[]) { int total = 0; for(int i=0; i<5; i++) total += arr[i]; return total; } </pre>	<pre> int sum2(int <u>*arr</u>) { int total = 0; for(int i=0; i<5; i++) total += arr[i]; return total; } </pre>
--	--

3

3

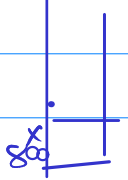
```

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int total = sum(arr);
    printf("%d", total);
    int total2 = sum2(arr);
    printf("%d", total2);
    return 0;
}
        
```

No difference

```

int x = 7;
sum(&x);
        
```



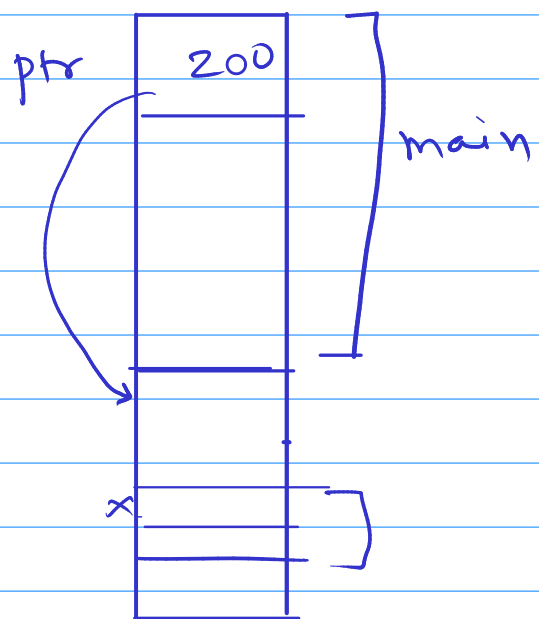
→ prefer [] when array is passed. otherwise prefer *

Que:5 Is following code valid? If no, why?

```

int *fun() {
    int x = 7;
    return &x;
}

int main() {
    int *ptr;
    ptr = fun();
    printf("%d", *ptr);
}
        
```



NOTE: Never return address of auto variable

→ Refrain from returning address of static local variable, even when it is allowed.

Que:6 Is following code valid? If no, why?
Yes

```
int *fun(int *ptr) {
```

```
    *ptr = 9;
```

```
    return ptr;
```

```
}
```

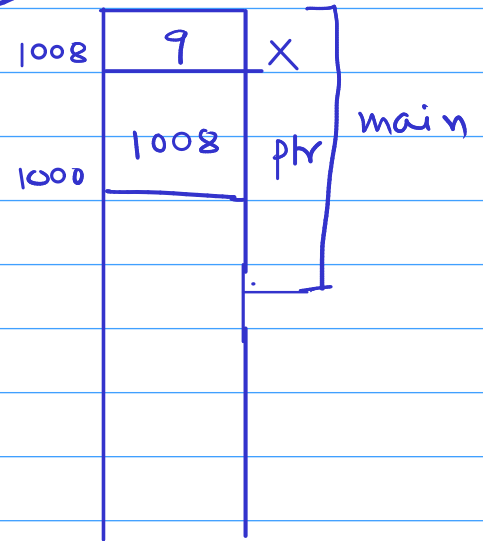
```
int main() {
```

```
    int X = 7, *ptr = &X;
```

```
    ptr = fun(&X);
```

```
    printf("%d", *ptr);
```

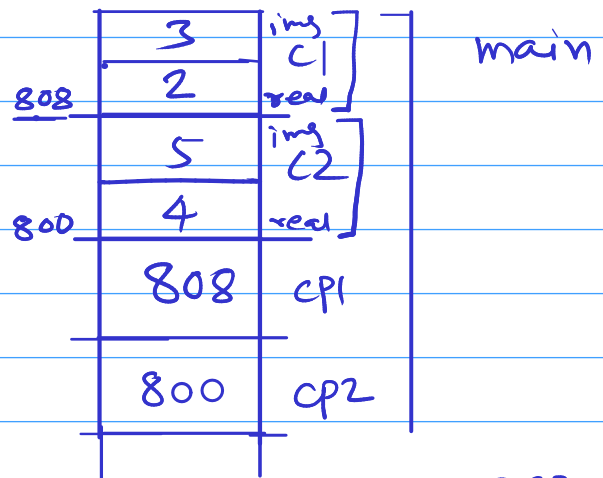
```
}
```



O/P 9

⇒ pointer to a structure variable

```
typedef
struct Cmplx {
    int real;
    int img;
} Complex;
```



```
int main() {
```

Same [Complex c1 = {2, 3}, *cp2;
struct Cmplx *cp1, c2 = {4, 5};

```
cp1 = &c1;
```

```
cp2 = &c2;
```

✗ cp1 = &(c1.real);

✓ int *ptr = &(c1.real);

H.W &c1.img

c1 & c2 are of same type
 cp1 & cp2 are of same type.

cp1 & cp2 are pointers to Complex type.

```
int x = (*cp1).real;
```

```
int x = cp1 → real;
```

arrow

operator → 1. deference operand on left side

2. Apply dot operator on deferenced variable

```
int main() {
```

```
    typedef  
    struct {
```

```
        int id;
```

```
        char name[20];
```

```
    } student;
```

```
    student s1 = {2, "Rohit"}, *sp = &s1;
```

```
X int id = sp.id;
```

```
    int id = sp -> id; // id will get value 2.
```

```
X char ch = sp -> name;
```

```
    char *ch = sp -> name;
```

```
    char ch1 = sp -> name[2]; // ch will get 'h'.
```

